

Domain_Driven_Design_Standard

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

1. Overview
2. Strategic Design
 - 2.1 Ubiquitous Language
 - 2.2 Bounded Contexts
 - 2.3 Context Mapping
3. Tactical Design
 - 3.1 Aggregates & Aggregate Roots
 - 3.2 Aggregate Design Principles
 - 3.3 Repository Pattern
 - 3.4 Domain Events
4. Package Structure
5. Legacy System Integration

Domain-Driven Design Standard

1. Overview

This document defines the practices for applying Domain-Driven Design (DDD) in the insurance technology platform, covering both strategic and tactical design. References include Eric Evans' original work and practices from Ant Group and Meituan.

2. Strategic Design

2.1 Ubiquitous Language

All project members communicate using a shared vocabulary maintained in the project Wiki. Key insurance terms:

Term	Meaning	Pinyin
Application	Insurance application submitted by customer (not yet effective)	Bao Dan
Policy	Effective insurance contract	Bao Dan
Servicing	Policy changes after issuance	Bao Quan
Underwriting	Risk assessment and decision process	He Bao
Claims	Insurance incident processing	Li Pei
Premium	Fee paid by policyholder	Bao Fei
Sum Insured	Coverage amount	Bao E
Deductible	Self-pay portion	Mian Pei E

2.2 Bounded Contexts

Insurance core domain contexts are organized into Core, Supporting, and Generic domains.

2.3 Context Mapping

Relationship	Pattern	Example
Collaboration	Partnership	UW ↔ Payment contexts
Shared Kernel	Shared Kernel	Policy ↔ Customer share customer model
Customer-Supplier	Customer-Supplier	Product → UW context
Separate Ways	Separate Ways	UW vs Claims, independent releases
Anti-Corruption	ACL	When integrating with legacy core systems

3. Tactical Design

3.1 Aggregates & Aggregate Roots

Context	Aggregate Root	Entities	Value Objects
Underwriting	Application	ApplicationItem, UWRecord	ApplicantInfo, RiskEvaluation
Policy	Policy	PolicyInsured, Beneficiary, Coverage	PremiumAmount, CoveragePeriod
Claims	Claim	ClaimItem, AssessmentResult	DamageAssessment, SettlementAmt

Context	Aggregate Root	Entities	Value Objects
Product	Product	Clause, Coverage, RateTable	RateFactor, CommissionRule
Payment	PaymentOrder	PaymentRecord, ReconciliationEntry	PaymentMethod, BankAccount

3.2 Aggregate Design Principles

1. Eventual consistency within aggregate, event-driven across aggregates
2. Reference across aggregates uses IDs, not object references
3. Transaction boundaries do not cross aggregate boundaries
4. One request modifies at most one aggregate
5. Aggregate size should fit within a single DB transaction

3.3 Repository Pattern

Each aggregate root has a corresponding Repository interface.

3.4 Domain Events

Key Domain Event	Trigger	Subscribers
PolicyIssuedEvent	Policy issuance	Channel, Reinsurance, Payment
PremiumPaidEvent	Premium received	Policy, Renewal
ClaimSettledEvent	Claim settlement	Payment, Reinsurance
PolicyLapsedEvent	Policy lapse	Renewal, Channel

4. Package Structure

```

com.company.insurance.underwriting
├── application
├── domain
│   ├── aggregate
│   ├── entity
│   ├── vo
│   ├── event
│   ├── repository
│   ├── service
│   └── spec
├── infrastructure
│   ├── persistence
│   ├── mq
│   └── client
├── interfaces
├── rest
├── rpc
└── job

```

5. Legacy System Integration

Legacy systems are adapted via the Anti-Corruption Layer (ACL) pattern:

New Service (DDD) → Anti-Corruption Layer → Legacy Core System

ACL responsibilities:

- Translate legacy data structures to new domain models
- Provide interface contracts expected by new services
- Isolate impact of legacy system changes